# Exercise - Introduction to OpenRoad



This exercise is still being updated for the 2026FS class. You can look at it but don't expect it to be in its final version until this message is displayed.

## Learning Objectives

- Experience the complete back-end flow in action
- Establish a reference design for the rest of the exercises
  - we will show students a design that has completed all design steps
- Learn how to navigate back-end tools

## TO DO list

- Fix typos
- Update PDK info and related tasks (e.g. metal stacks)
- Adapt chip size
- Switch to new images
- Give an overview of the technology files required and their purpose (e.g. by explaining the `init_tech.tcl` script)
- Add section about checkpointing throughout design flow (see `checkpoint.tcl`)

## Introduction

In this exercise you will explore the back-end design flow and get familiar with using the leading open-source back-end tool OpenROAD. It is meant as a first introduction to OpenROAD, in which you will familiarize with the tool's GUI and TCL console. For this sake, we will provide you with a completed back-end design flow of an actual manufactured ASIC, the Croc SoC. You will use this as a reference to familiarize with the back-end tool and explore its functionality.

### About the style

We will use a number of different styles to identify different types of actions as shown below:

**Student Task**

Parts of the text that have a gray background, like the current paragraph, indicate steps required to complete the exercise.

Actions that require you to select a specific menu will be shown like the following: `menu → sub-menu → sub-sub-menu`

Whenever there is an option or a tab that can be found in the current view/menu we will use a `BUTTON` to indicate such an option.

Throughout the exercise you will be asked to enter certain commands using the command-line.

```
sh> command to be entered on the Linux command line
```

***Note:*** *there are many reasons for using a command-line. Some functionality can not not be accessed through GUI commands, and in some cases, using the command-line will be much faster. Most importantly, things you enter on the command-line can be converted into a script and executed repeatedly.*

## Preparation

**Student Task 1**

**For ETH users:**

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex05
sh > cd ex05
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex05/install.sh`.

**For non-ETH users:**

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex05
```

# Using OpenROAD

First a warning: OpenROAD is the state-of-the-art open-source backend design tool. As such it contains many features that are not necessary for these exercises. Additionally, many features require you to know or look up commands, there are no GUI options for them (to some extend this is also the case for commercial tools).

To get familiar with OpenROAD, let us take a look at a completed design first.

### Student Task 2

- Initialize the exercise directory by running the Cockpit tool. This will create a predefined directory structure and copy the required technology for the IHP13SG2 process node files into the `technology` sub-directory. Please refer to the Exercise setup page for more details on the Cockpit tool. Make sure to run the `icdesign` command in the same directory where the `.cockpitrc` configuration file is located.

```
sh (exx05) > cd croc
sh (croc)  > icdesign ihp13 -update all -nogui
```

- Start OpenROAD through the Cockpit GUI button.
- Alternatively, you can launch OpenROAD from the terminal, by running the following command from the *openroad* directory:

```
sh (croc)     > cd openroad
sh (openroad) > oseda -2025.01 openroad -gui
```

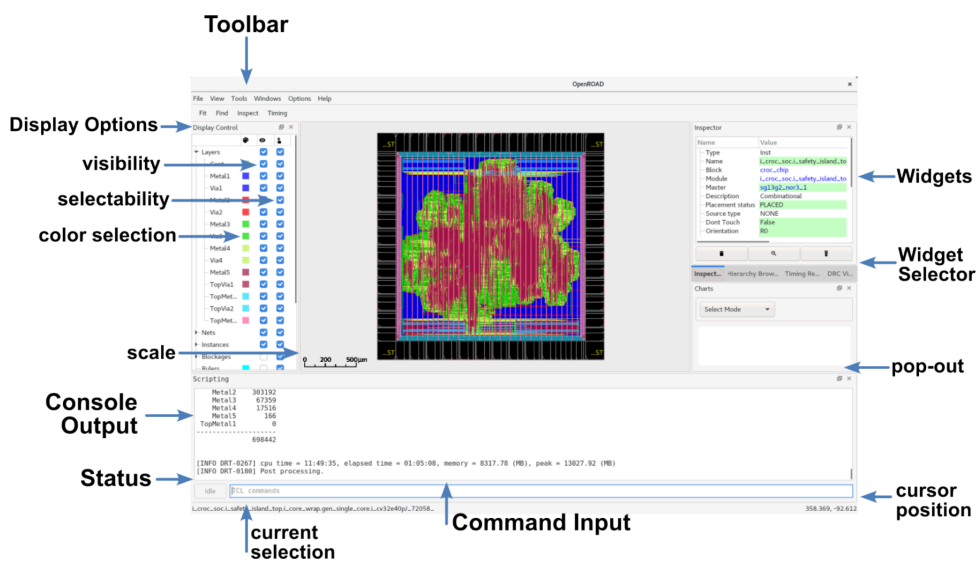

**Figure 1**: Important elements of an OpenROAD window.

The OpenROAD GUI should appear. The most important part is the 'Scripting' pane at the bottom. Here you can enter commands and it shows you their outputs (the same output is also in the terminal window you started OpenROAD with). You can pop it out or re-arange it using the symbol with the two windows on the right side of the 'Scripting' pane. You will be using the console more than you think. Make sure that it remains accessible and visible at all times! If you accidentally close it, you can open it via the toolbar at the top by selecting *Windows→ Scripting*. Similarly if you decide to start OpenROAD without the gui (without the *-gui* flag), you can open the gui using:

```
OpenROAD > gui::show
```

Note also that while commands are running, the GUI will be frozen. So if you think OpenROAD has crashed, first check the console output and your terminal for activity.

# Terminology

Before we start to explore the example design, some notes about the terminology:

**Standard Cell / Cell** Building block that implements a logic gate (and, or, flip-flop, latch, etc.) and will be placed (aligned in rows) in the design core area.

**Module** It is equivalent to the entities in VHDL, or modules in Verilog. It contains a level of the design hierarchy.

**Instance** One specific copy of a building block that is part of your design netlist. Identified by a unique instance name, e.g. `provide example instance`. Note: also used for Verilog modules used in your design netlist.

**Macro-cell / Block** Building block that is typically larger and more complex than the standard cells. This could be a sub-design that has been completed previously or a full-custom block like the RAMs used in this example. Typically placed somewhere in the core area.

**Pad** Building block that is used to connect the core of the chip to the external world. Placed in the I/O region around the core.

**Master** A building block available from a library, e.g. 'RM_IHPSG13_1P_256x48_c2_bm_bist', 'sg13g2_and2_1', 'sg13g2_inv_2' etc. Note: Also used for Verilog modules defined in your design netlist.

**Row** All standard cells have the same height. This allows them to be placed in (horizontal) rows. In back-end terms, a row defines a region where standard cells can be placed.

**ITerm** A logical connection point (port) of a module or cell, e.g. the inverter 'sg13g2_inv_1' has two terminals ('A' and 'Y').

**<Inst> Pin** A physical connection point/shape of an instance, i.e. where the router contacts the instance.

**Net** The logical/signal interconnection between instances.

**Special Net** All instances need to be connected to power and ground. Technically these connections are also nets, but they are treated differently from regular nets. These nets are called special nets.

# About the technology

This series of exercises is based on IHP's open-source PDK for analog/digital, mixed-signal and RF design. The SG13G2 (https://github.com/IHP-GmbH/IHP-Open-PDK) process node belongs to a family 130nm SiGe BiCMOS technologies and provides a fully open-source PDK that can be manufactured at IHP's facilities. According to the documentation, this process node provides two gate oxides: A thin gate oxide for the 1.2 V digital logic and a thick oxide for a 3.3 V supply voltage. For both modules NMOS, PMOS and isolated NMOS transistors are offered. Further passive components like poly silicon resistors and MIM capacitors are available. The backend option offers 5 thin metal layers, two thick metal layers (2 and 3 µm thick) and a MIM layer.

Although the technology is open-source and you can easily access it on Github (https://github.com/IHP-GmbH/IHP-Open-PDK), on ETHz workstations you can access different pre-installed versions by means of the Cockpit (http://eda.ee.ethz.ch/index.php?title=Cockpit) tool. Note that in these exercises you will be using a later revision of the PDK, which is not open-source yet. While most aspects of the design are largely unaffected by this choice, the newer revision includes some minor changes to the process that ensure a better quality of the result. If you are working on these exercises from outside ETH, you can use the open-source version of the PDK instead.

# Exploring the design

Now let us restore a completed database. As for the past exercises, we use the Croc SoC as our reference design. Although the following exercises will focus on this design for simplicity, all the steps of the back-end flow that we will cover are independent of the design, and could be applied to other designs as well. One of the advantages of using a simple design is the short flow turnaround time. Backend flows for large designs can take weeks to complete, so do not expect the same experience when exploring more complex designs. Nevertheless, the concepts and scripts that you will learn in the next exercises will apply for the major part to larger designs as well.

For our flow we use a script to create checkpoints after all major steps. It writes out all relevant files and zips them together. This allows us to trace down issues we encounter in the flow.

The most important files are the following:

- **Database**: files ending in `.odb` contain OpenROAD's database format of the design. It includes the physical design along with the netlist in its internal format.
- **Netlist**: files ending in `.v` are design netlists. They are not necessary for reloading a design but they are very convenient to explore what changed compared to the last checkpoint. At any point during this exercise you may want to open the netlist and compare it with what you are seeing in OpenROAD.
- **Constraints**: files ending in `.sdc` are Synopsys Design Constraints. They describe timing limitations the design must meet at any point. Most importantly, they contain the definition of all clocks used. After loading the database you may want to load the constraints (check the `read_sdc` command) in order to restore timing information.
- **Technology**: files used to describe the functionality and physical design of all the cells and macros used in the design, as well as the geometry of the metal layers used to connect cells together. Technology files do not change throughout the backend flow, and are therefore not saved in the flow checkpoints. You can reload the technology information at any point using the script `scripts/init_tech.tcl`.

### Student Task 3

- Select *File→ Open DB* from the menu. The design is stored in OpenROAD's open-database format, it is stored as 'croc.odb' inside the 'out' directory.

After loading the database you will see a routed chip similar to Figure 2.

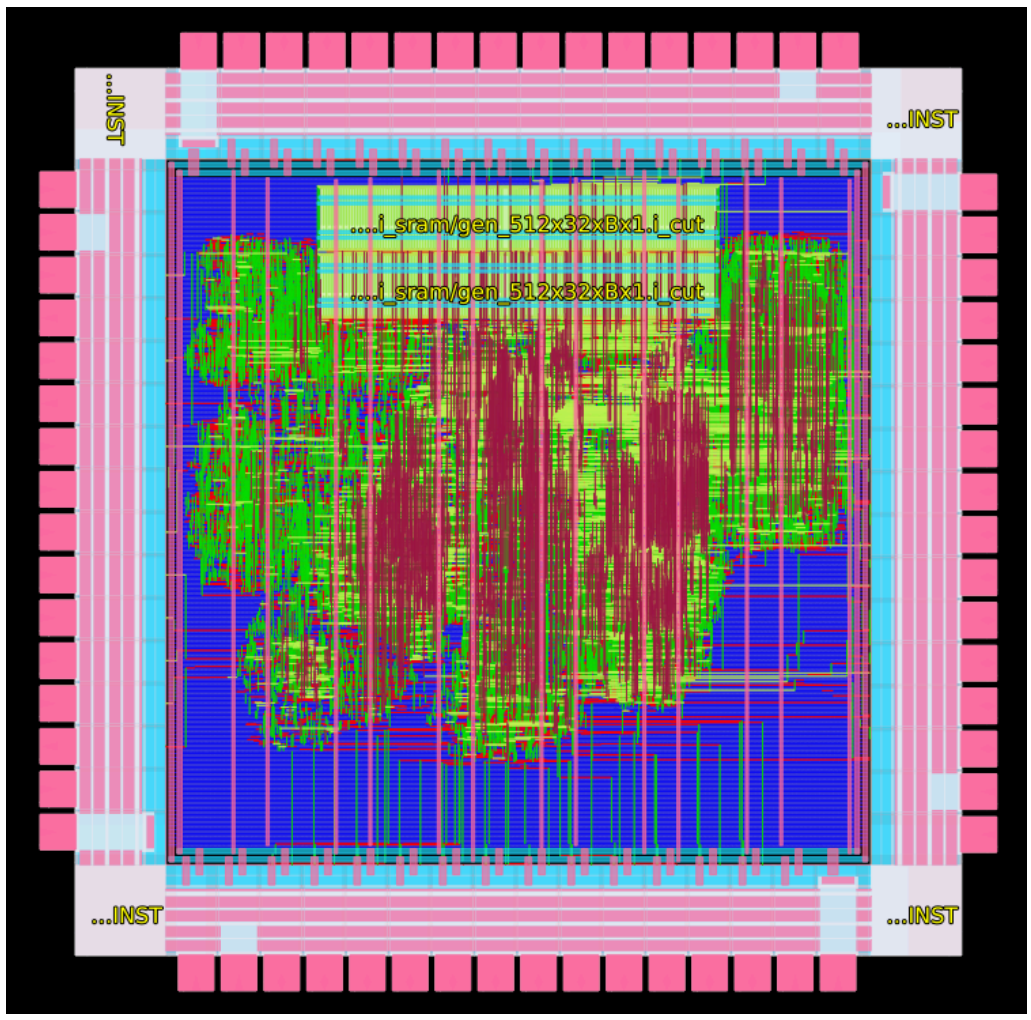


**Figure 2**: Croc chip loaded in OpenROAD.

## Basic Navigation

You can zoom in by using the mouse wheel while holding the *Ctrl* key, or by drawing a rectangle with the mouse right button. If you want to go back to seeing the full chip, either use 'Fit' from the lower toolbar or with the keyboard shortcut (***F*** key). You may also notice that some letters in the toolbar are underlined (e.g. V for 'View'), you can press *Alt* and the highlighted letter to open this toolbar menu, then use up and down keys to navigate the dropdown.

Pan by using the mouse wheel (up and down) or by pushing the mouse wheel and moving the cursor. By holding *SHIFT* while moving the mouse wheel, you can scroll up and down by one screen at a time.

Press the mouse right button to reveal menu with some useful options. Most importantly, the *Clear* submenu allows you to clear highlights, selected objects, or the rulers currently displayed. You can remove all the rulers by pressing *SHIFT* + *K*.

### Student Task 4

- To get a better impression of the size, let us try to measure the chip dimensions. First we will have to select the *ruler*tool from the toolbar in *Tools→ Ruler* or by using its keyboard shortcut *K*. The units are in micrometers. How long are the sides of the chip?

```
h = _______________
w = _______________
```

- When you look at the chip, you realize that the area used for the pads/IO-cells is not very small. The area in the center is called core area, and in between the core area and the pad ring is a set of power wires called the core power ring. What is the ratio between core area (with power ring) and the total area?

```
Atotal/Acore = ______________
```

- There are two identical SRAM blocks placed in this design. What is the area of such a RAM module? In this example, the RAM holds 512 words of 32 bit each. What is the bit density in this technology, in other words how many bits can you store per squared millimeter ?

```
A = _______________
d = _______________
```

## Display Controls

The design we have loaded, is already routed and you can see a jungle of wires on the metal layers. You will need to zoom in really close to differentiate individual connections. A modern IC manufacturing technology provides several metal layers for routing, the technology we use for these exercises has 5+2 (two are `TopMetal` which are significantly thicker). Each metal layer is separated by an insulating layer. In this way separate signals can be carried on different metal layers. To move a signal from one metal layer to another a special connection named *via* is used. Each via connects two adjacent metal layers. In this technology multiple vias can also be stacked (placed on top of each other), so that it is possible to move from the top most metal (`TopMetal2`) to the bottom most metal layer (`Metal1`) at one point [1].

There are several colored squares in *Display Control* on the left hand side of the GUI. Each square shows the color of an object category in the design. The object category is written on the left side of the colored square. On the right side of the squares you can see two check boxes. The first check box is used to toggle the visibility of an object category. Let us take the object category *Nets*. This category contains the various type of connections (power (*Power* for `Vdd` and *Ground* for `Vss`), clock and data/signals). Click on the first check box of *Signal* so that the box is not selected. You will see all data interconnections disappear. Only the clock tree and the power distribution remains. Turn off visibility for *Power* and Ground as well, you can now see the placed standard cells in light-gray with the clock tree on top. The second box determines whether or not you will be able to select objects belonging to that category. Remember that you can only select objects that are visible. I recommend you turn off selectability for *Power* and *Ground*, it is very easy to accidentally click on them and select this very large net (turning off `Metal1` also helps a lot with this and is a good alternative).

We will not explain all object categories (you can explore it yourself) but we highlight the most important ones. The category *Instances* will control all instances (*Cells*, *Pads*, *Macros*,. . . ) in your design, and *Misc/Instances* controls various parts of how instances are shown. *Misc/Highlight selected* controls highlighted objects and *Heat Maps* contains various different overlays.

### Student Task 5

- There are 5+2 metal layers in this process. Typically, each metal layer is used predominantly in one direction (either horizontal or vertical). For the metal layers 2 to 5, find out in which direction it is running.

```
You will need the section Layers in the layer control area on the right to switch the physical layers on and off.
M1  _______________
M2  _______________
M3  _______________
M4  _______________
M5  _______________
TM1 _______________
TM2 _______________
```

- When looking at the individual metal layers, do you notice any exceptions in the direction of the wiring? Why or why not?

_____

- Follow the power connections of the SRAM. On which metal layer is the connection, where does it connect to?

```
 Hint: You can switch off the visibility of the majority of connections (Nets) to see only power and ground. Additionally,
you can hide all instances except Macros (Instances/Macros) You might also want to toggle the pins of the RAM block
(Misc/Instances/Pins).
```

_____

- Switch on a single metal layer in vertical direction, e.g. M3. You should notice a very distinctive, regular pattern all across the chip (you need to make all *Nets* visible again if you have switched it off before). You probably have noticed that the main power distribution grid is on layers TM1 and TM2 with very wide tracks on TM2. Yet it clearly also seems to influence the routing on other metal layers. Can you explain this?

_____

## Overlays

OpenROAD has various different overlays you can activate. Each is designed to give you further insight into your design. The first one we will look at it is the *Module* view. It shows you where certain blocks of your design are placed. Typically each module in your RTL performs a smaller part of the overall chips functionality. This means the logic in these modules is closely related to each other. In the chip this often means they form large blobs (commonly called amoeba) of standard cells. This is exactly what the *Module view* shows you.

### Student Task 6

- Continuing from before, besides the vertical pattern, it seems that there are clear borders between different clusters of routing. If you turn on Metal5 you can also see that its only used in certain locations. To analyze the source of this, in the Widget view select *Hierachy Browsers* on the right of the main window. Then click on *Module view is no enabled*, to enable this view (you can also find it under *Misc/Module view*).

```
 You can select and de-select each module or change their colors. Zoom in to various areas and toggle Misc/Module view on
and off.
 Can you now explain these clear borders in the routing layers? Routing is divided into 'blocks', why?
 _____
```

Related to the exploration of wiring is the concept of 'routing congestion'. Same as on the street, congestion is bad especially if it crosses a certain threshold and becomes too big to handle. Routing congestion tells us how many vertical and horizontal routing resources are used in some area. You can display the associated heat map by selecting *Heat Maps/Routing Congestion*. But first, I recommend you turn off visibility of `Power` and `Ground` nets to get a cleaner overlay. To make it a bit easier to read, we can also change the heat map settings under *Tools→ Heat maps→ Routing Congestion*.

We recommend you turn on *Show numbers* (displays the percentage in each grid-cell when zoomed in) and *Show legend*. Play with the `Minimum` and `Maximum` until you find values that give you a nice picture. Note: You probably want to make sure *Misc/Module view* is turned off, otherwise the color will bleed through to the overlay.

### Student Task 7

- Compare the heat map with the routing on Metal5, do you notice something?
- OpenROAD also has a congestion estimation algorithm called RUDY. Turn on its heat map instead and compare the two. Do they match?

# Instance Inspector & Search

Using the *Hierarchy Browser*, the *Inspector* (also a widget) and the *Find* function (top-left or *CTRL+F*) you can browse through the logical hierarchy of the design, highlight and select instances and nets. To find objects using the search command it is also possible to use wild cards (* and *?*: the * matches any number of characters, a *?* can be used to match exactly one character).

However, often you need to do more than just find instances, instead you need to be able to see their connectivity to the next few components. We examine one option to do so with the next task.

### Student Task 8

- We start at an SRAM, click on one and it will populate the *Inspector* widget on the right. Clicking on the blue text in the *Inspector* will change the selection to this object.
- Click/hover on the 'Master' cell. This should select all SRAMs.
- Click on any instance in the selection shown in the *Inspector*, we are back at just one SRAM.
- Expand 'ITerms', this shows you all SRAM pins (left) and the connected net (right).
- Click on one of the nets connected to a data pin. The net is now selected (shown in yellow).
- At the bottom of the Inspector you have various buttons. Try them out, what does each one do?
  - How can you zoom in to the net?
  - How can you add it to a highlight group?
  - How can you show only this net?
- Navigate to a pin of some instance (select some instance and navigate to the correct *ITerm*, or display the Pins of instances, zoom-in and click directly on a pin).
- Display all connections to/from this pin using the button of the *Inspector*. Using this newly acquired knowledge, try to figure out why two SRAMs are always placed on-top of each other.

```
Hint: Explore the connectivity of the data-pins into the SRAM and the cells driving them.
_____________________________________________________________________________________________
```

- Somewhere in the design is a cell called something-something-i_clkgate, find it and figure out in which module it is.

```
_____________________________________________________________________________________________
```

# Clock Tree

Another important part of any design are the clocks. They are designed in a special way to make sure all the flip-flops receive the clocks at the same time. In particular they are implemented using tree structures, hence the name *Clock-Tree*.\ OpenROAD can give us various views of the clock-tree but first we need to actually tell it where our clocks are. For this we need to load a constraints file where we define the clocks (among other things). However, before we can do so we should first reload the technology files to make sure everything is present and configured properly.

### Student Task 9

- Load the technology data using (this may give some 'already exists' errors, ignore them).

```
OpenROAD > source scripts/init_tech.tcl
```

- Now load the constraints for this design.

```
OpenROAD > read_sdc save/croc.drt/croc.drt.sdc
```

- Finally, open the Clock Tree Viewer widget and click on Update.

You should now be looking at a schematic view of the clock tree(s) in the widget as well as having the clock tree highlighted on the chip. The highlighted nets are divided into two colors, they always correspond to the left and right branch shown in the schematic view.

### Student Task 10

- Navigate to different buffers in the schematic view and click on them, what is shown in the chip view?
- Try to find i_clkgate again in the schematic view and select the branch of the clock-tree it is gating.

## Further Help

These exercises will only explain the basic functionality of OpenROAD, for additional help please use the *Help* function in the toolbar. It will lead you to the OpenROAD documentation, where you can find additional information. Newer versions of OpenROAD also have a *man* command to show you helpful information about commands. The OpenROAD version we are using does not yet have this functionality so you will have to use the online documentation (https://openroad.readthedocs.io/en/latest/index.html) instead.

## A last note

Back-end design is an iterative process. The decisions you will make at the beginning of back-end design may have a profound impact on the outcome. However, you will not always be able to predict this. You will have to complete individual stages of back-end design (floorplaning, placement, routing etc) and then assess the quality of the result. If you are not satisfied with this result you will have to repeat the stage or even the entire process. This is normal, be prepared for it. Notice that each time you repeat a certain stage, you will gain more experience and will be able to work faster.

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).

1. Note that there will be 6 separate vias that are placed on top of each other for this to happen. This will block all in-between metal layers (Metal2, Metal3, . . . , TopMetal1) at this point as well.